手写作业自动批改系统 - 编码规范文档

1. 通用原则

1. **代码可读性优先**：代码是写给人看的，其次才是给机器执行的。命名应清晰表达意图，避免过度缩写。
2. **单一职责原则 (SRP)**：每个类、函数只负责一项明确的任务。
3. **防御性编程**：对所有外部输入（文件、API 参数、数据库数据）进行校验，严禁信任前端或下游服务传来的数据。
4. **日志规范**：生产环境禁止使用 `System.out.println` 或 `print`，必须使用日志框架，且需包含上下文信息（如 `submissionId`, `userId`）。
5. 异常处理
   :
   - 严禁捕获异常后不做任何处理（吞掉异常）。
   - 业务异常抛出自定义异常或明确的 `RuntimeException`。
   - 系统异常需记录堆栈 (`exc_info=True` / `log.error(..., e)`)。

---

2. Java 后端编码规范 (Spring Boot)

2.1 命名规范

- **类名**: `UpperCamelCase` (例: `SubmissionService`, `GradingResult`)。
- **方法/变量**: `lowerCamelCase` (例: `gradeSubmission`, `totalScore`)。
- **常量**: `UPPER_CASE_WITH_UNDERSCORES` (例: `MAX_FILE_SIZE`, `ROLE_STUDENT`)。
- **包名**: 全小写，单数形式，层级清晰 (例: `com.homework.service`, `com.homework.entity`)。
- **DTO/VO**: 后缀明确，如 `LoginRequest`, `LoginResponse`, `AssignmentVO`。

2.2 代码风格

- **Lombok 使用**: 实体类 (`Entity`) 和 DTO 必须使用 `@Data`, `@Builder`, `@AllArgsConstructor` 等注解减少样板代码，但需注意 `equals/hashCode` 在 JPA 实体中的潜在问题（建议 Entity 仅用 `@Data` 或手动指定字段）。
- **集合处理**: 优先使用 Java Stream API 处理集合转换，但避免过度嵌套导致可读性下降。
- 魔法值
  : 严禁在业务逻辑中直接出现数字或字符串字面量。
  - ❌ `if (status == 1)`
  - ✅ `if (status == AssignmentStatus.PUBLISHED.getCode())` (建议使用枚举或常量类)。
- BigDecimal 运算
  : 涉及金额、分数必须使用

  ```
  BigDecimal
  ```

，严禁使用

```
double/float
```

。运算时必须指定

```
RoundingMode
```

。

- ✅ `score.multiply(ratio).setScale(2, RoundingMode.HALF_UP)`

2.3 异常与事务

- **事务控制**: `@Transactional` 应加在 Service 层方法上。注意事务粒度，避免在事务内执行耗时 IO 操作（如 HTTP 请求、大文件读写），本项目中 OCR 调用已异步化，符合规范。
- **全局异常处理**: 建议补充 `@RestControllerAdvice` 统一捕获 `RuntimeException`，返回标准化的 `R<T>` 格式（当前代码中部分 Service 直接抛 RuntimeException，需确保 Controller 层或全局处理器能正确捕获并转换为 `R.fail()`）。

2.4 数据库交互 (MyBatis-Plus)

- 查询构造

  : 强制使用

  ```
  LambdaQueryWrapper
  ```

  ，避免硬编码字段字符串。

  - ✅ `new LambdaQueryWrapper<User>().eq(User::getUsername, name)`
- **批量操作**: 避免在循环中单条插入数据库，应收集列表后批量处理（若 MP 版本支持 `saveBatch`）。

2.5 安全规范

- **密码存储**: 必须使用 BCrypt 加密 (`PasswordEncoder`)，严禁明文存储。
- **SQL 注入**: 严禁使用字符串拼接 SQL，必须使用 `#{}` 占位符或 MP Wrapper。
- 文件上传

  :

  - 必须校验文件后缀名。
  - 文件名必须重命名（UUID + 时间戳），严禁使用用户原始文件名保存。
  - 限制文件大小（已在配置中限制，代码层建议二次校验）。

---

3. Python 微服务编码规范 (Flask)

3.1 命名规范

- **模块/文件**: `snake_case` (例: `image_processor.py`, `ocr_engine.py`)。
- **类名**: `UpperCamelCase` (例: `MockOcrEngine`)。
- **函数/变量**: `snake_case` (例: `preprocess_bytes`, `all_texts`)。
- **常量**: `UPPER_CASE` (例: `MAX_CONTENT_LENGTH`)。

3.2 类型提示 (Type Hinting)

- 所有函数必须包含参数类型注解和返回值类型注解。
  - ✅ `def preprocess_bytes(image_bytes: bytes) -> np.ndarray:`
  - ✅ `def recognize_by_region(...) -> Dict[int, str]:`
- 使用 `typing` 模块处理复杂类型 (`List`, `Dict`, `Optional`, `Tuple`)。

3.3 图像处理规范

- **内存管理**: 处理大图片时注意内存释放。OpenCV 操作尽量在原图或副本上进行，避免不必要的深拷贝。
- 异常容错

  : 图像预处理步骤（如倾斜校正）若失败，应返回原图或降级处理，严禁导致整个服务崩溃。
  - ✅ `try...except Exception: return binary_image` (见 `deskew` 函数)。
- **魔数管理**: 图像处理参数（如高斯核大小 `(3,3)`，阈值 `11, 2`）应提取为模块级常量或配置项，并添加注释说明其物理含义。

3.4 依赖管理

- **延迟加载**: 重型库（如 `paddleocr`）必须在函数内部或懒加载模式中导入，防止服务启动超时或环境缺失导致崩溃（当前 `get_ocr_engine` 实现符合规范）。
- **虚拟环境**: 必须使用 `requirements.txt` 锁定依赖版本，禁止在生产环境使用最新版未测试依赖。

3.5 接口规范

- **响应格式**: 所有 API 必须返回统一的 JSON 结构 `{code, message, data}`。
- **状态码**: 严格遵循 HTTP 语义 (200 OK, 400 Bad Request, 500 Internal Error)。
- **CORS**: 仅允许受信任的域名访问（生产环境应配置 `CORS(app, resources={r"/api/*": {"origins": "..."}})`，当前 `CORS(app)` 过于宽松，需整改）。

---

4. 日志规范

4.1 日志级别使用

- **ERROR**: 系统错误、异常堆栈、外部服务调用失败（如 OCR 超时）。必须包含堆栈信息。
- **WARN**: 可恢复的异常、参数解析失败、降级逻辑触发（如 regions 解析失败）。
- **INFO**: 关键业务流程节点（如 "OCR 识别完成", "批改开始", "用户登录成功"）、服务启动信息。
- **DEBUG**: 详细变量状态、中间处理结果（生产环境通常关闭）。

4.2 日志内容

- 必须包含**业务上下文 ID**（如 `submissionId`, `assignmentId`, `userId`），便于链路追踪。
- ❌ `logger.error("OCR failed")`
- ✅ `logger.error(f"OCR 识别失败 submissionId={submissionId}, error={str(e)}", exc_info=True)`

4.3 敏感信息

- **严禁**在日志中打印用户密码、完整 Token、个人隐私信息（手机号、身份证）。

---

5. 项目结构与文件组织

5.1 Spring Boot

text

```
src/main/java/com/homework
├── common          # 通用类 (R, Constants, Exceptions)
├── config          # 配置类 (SecurityConfig, AsyncConfig, CorsConfig)
├── controller      # 控制层 (仅处理参数校验和响应封装)
├── service         # 业务逻辑层 (接口 + 实现)
├── mapper          # DAO 层
├── entity          # 数据库实体
├── dto             # 数据传输对象
└── security        # 安全相关 (JwtUtil, Filter, Principal)
```

5.2 Flask

text

```
ocr_service/
├── app.py           # 入口文件
├── image_processor.py  # 图像处理逻辑
├── ocr_engine.py       # OCR 引擎封装
├── requirements.txt
└── logs/               # 日志目录
```

---

6. 代码审查 (Code Review) 检查清单

在合并代码前，必须确认以下事项：

1. **[Java]** 是否所有涉及分数的计算都使用了 `BigDecimal`？
2. **[Java]** 是否存在事务中包含 HTTP 远程调用或文件 IO 的情况？（应异步化）
3. **[Java]** 数据库查询是否避免了 `N+1` 问题？
4. **[Python]** 图像预处理是否有边界条件检查（如空图、极小图）？
5. **[Python]** 是否移除了所有的 `print` 语句？
6. **[通用]** 敏感配置（DB 密码、JWT Secret）是否已从代码中移除并放入环境变量/配置文件？
7. **[通用]** 异常捕获后是否记录了有意义的日志？
8. **[通用]** 接口是否通过了基本的单元测试或集成测试？

---

7. 附注：针对当前代码的特定改进建议

基于提供的代码片段，以下具体点需在后续开发中严格遵守或修正：

1. **Flask CORS 配置**: 当前 `CORS(app)` 允许所有域，生产环境必须限制为 Spring Boot 的地址。
2. **Java 异常统一处理**: 当前 `AuthService` 和 `SubmissionService` 直接抛出 `RuntimeException`，需确保有全局 `@RestControllerAdvice` 将其捕获并转为 `R.fail()` 格式，否则前端会收到 HTML 错误页而非 JSON。
3. **关键词解析逻辑**: `GradingService` 中 `keywords` 的解析方式 (`replace`) 较为脆弱，若关键词本身包含逗号或引号会出错。建议存储为标准 JSON 字符串，使用 `JSON.parseArray` 解析。
4. 并发安全

   :

   ```
   SubmissionService.processAsync
   ```

   中使用的

   ```
   MultipartFile
   ```

   在异步线程中可能因请求结束而被清除。

   - **修正**: 必须在主线程中将 `MultipartFile` 读取为 `byte[]` 或保存到临时文件，再将字节/路径传递给异步方法。当前代码直接传递 `imageFile` 对象存在高风险（取决于 Spring 配置，通常 `MultipartFile` 依赖于请求流，异步时流可能已关闭）。**建议立即修改为传递 `byte[]` 或文件路径。**

5. **Mock 引擎线程安全**: `MockOcrEngine` 无状态，目前安全。若未来引入有状态的引擎，需注意全局变量 `_ocr_engine` 的初始化锁。